

Sistemas Operativos II – Practica 2

Resolución del problema con threads

Nuria Guerra Casal Adrian Gijon Yubero
Grao en Enxeñaría Informática

1. Introducción

Este ejercicio resuelve el mismo problema productor-consumidor del apartado 2 (`prod_sem.c` y `cons_sem.c`), pero sustituyendo los dos procesos independientes por *threads* (`pthread`) dentro de un único proceso. El objetivo es comprender que diferencia existe entre ambos modelos de concurrencia y verificar que los semáforos funcionan igual en los dos contextos.

La diferencia fundamental es el espacio de memoria: dos procesos tienen espacios separados y necesitan mecanismos como `mmap()` para compartir datos, mientras que los *threads* de un mismo proceso comparten automáticamente toda su memoria. Esto simplifica la implementación, pero **no elimina la necesidad de sincronización**: sin semáforos, dos *threads* que acceden concurrentemente al mismo buffer seguirían produciendo carreras críticas exactamente igual que en el apartado 1.

2. Diferencias respecto al apartado 2

Aspecto	Procesos (apartado 2)	Threads (ejercicio 1)
Memoria compartida	<code>mmap()</code> sobre fichero	Variable global directa
Crear semáforos	<code>sem_open(..., O_CREAT)</code>	<code>sem_init(&sem, 0, val)</code>
Abrir semáforos	<code>sem_open(..., 0)</code> en el consumidor	No necesario
Eliminar semáforos	cierre y borrado explícito	<code>sem_destroy(&sem)</code>
Lanzar ejecución	2 ejecutables	<code>pthread_create()</code>
Esperar finalización	Espera activa	<code>pthread_join()</code>
Terminales necesarios	2 terminales separados	1 único terminal

Al compartir memoria, el buffer y los semáforos se declaran como **variables globales**, accesibles directamente por ambos *threads* sin ninguna configuración adicional. Los contadores de vocales también son globales para que el `main()` pueda imprimirlos ordenadamente tras el `pthread_join()`:

```
1 DatosCompartidos datos; //buffer LIFO compartido (sin mmap)
2
3 sem_t vacias; //huecos libres (inicial = TAMANO_BUFFER)
4 sem_t cheas; //elementos disponibles (inicial = 0)
5 sem_t mutex; //exclusion mutua (inicial = 1)
6
7 // Contadores globales para imprimirlos en main() tras pthread_join()
8 int prod_a=0, prod_e=0, prod_i=0, prod_o=0, prod_u=0;
9 int cons_a=0, cons_e=0, cons_i=0, cons_o=0, cons_u=0;
```

Listing 1: Variables globales compartidas entre threads

3. Inicialización y destrucción de semaforos

Con *threads* se usa `sem_init()` en lugar de `sem_open()`. La diferencia clave esta en el segundo parametro, `pshared`: si el valor es 0, el semáforo existe solo en memoria del proceso y es accesible únicamente por sus *threads*; si el valor es 1, el semáforo sería compartible entre procesos distintos, pero requeriria `mmap()`. Al no tener nombre en el sistema de ficheros, no hace falta `sem_unlink()` al terminar, basta con `sem_destroy()`.

```
1 // pshared=0: solo accesible por threads del mismo proceso
2 sem_init(&vacias, 0, TAMANO_BUFFER); //huecos libres = N
3 sem_init(&cheas, 0, 0); //elementos disponibles = 0
4 sem_init(&mutex, 0, 1); //mutex libre = 1
5
6 // ... ejecucion ...
7
8 sem_destroy(&vacias); //no hace falta sem_unlink(), no hay nombre en el sistema
9 sem_destroy(&cheas);
10 sem_destroy(&mutex);
```

Listing 2: Inicialización con `sem_init()` y liberación con `sem_destroy()`

4. Funcionamiento de las funciones principales

Las funciones `insert_item()` y `remove_item()` siguen el mismo protocolo de semáforos que en el apartado 2. En `remove_item()`, además de retirar el carácter, se sustituye la posicion por '-', y se imprime el estado del buffer:

```
1 char remove_item(void) {
2     sem_wait(&cheas); //espera elemento disponible
3     sem_wait(&mutex); //exclusion mutua
4
5     datos.cima--;
6     int pos = datos.cima;
7     char c = datos.pila[pos];
8     datos.pila[pos] = '-'; // sustituye por '-'
9
10    //imprime estado del buffer para verificar el funcionamiento
11    sem_post(&mutex);
12    sem_post(&vacias); //avisa al productor de hueco libre
13    return c;
14 }
```

Listing 3: `remove_item()`: retira el carácter y sustituye por guión

Los conteos de vocales se acumulan en variables globales durante la producción y el consumo, y se imprimen en el `main()` despues de que ambos threads hayan terminado con `pthread_join()`, garantizando que los valores son definitivos.

5. Velocidades y fases de ejecución

Los `sleep()` se colocan fuera de la región critica, siguiendo las tres fases:

Iteraciones	Productor	Consumidor
0 – 29	Sin sleep (rapido)	Sleep 3s (lento) → buffer se llena
30 – 59	Sleep 3s (lento)	Sin sleep (rapido) → buffer se vacia
60 – 79	Sleep aleatorio 0–3s	Sleep aleatorio 0–3s

6. Ejecución

```
# Compilar (-pthread es obligatorio para pthread y sem_init)
gcc -o threads threads.c -pthread

# Ejecutar (un solo terminal, un solo ejecutable)
./threads texto.txt
```

A diferencia del apartado 2, no es necesario coordinar el inicio de dos programas en terminales distintos. Los dos *threads* arrancan automáticamente al lanzar el programa y el `main()` espera su finalización antes de imprimir los resultados.

7. Resultados y conclusiones

Durante la primera fase la salida muestra ráfagas de mensajes [Productor] sin interrupciones, hasta que el buffer llega a `cima=10` y el productor queda bloqueado en `sem_wait(&vacias)`. A partir de ese momento sólo aparecen mensajes cada vez que el consumidor libera un hueco (cada 3 segundos). Durante la segunda fase ocurre lo contrario: el consumidor vacía el buffer rápidamente y queda bloqueado en `sem_wait(&cheas)` entre cada producción del productor.

```
[Productor] Le: 's'
[Productor] Inserido 's' en pila[5] cima=6
[Productor] Pila: [Ese es----] cima=6

[Productor] Le: ' '
[Productor] Inserido ' ' en pila[6] cima=7
[Productor] Pila: [Ese es ---] cima=7

[Productor] Le: 'u'
[Productor] Inserido 'u' en pila[7] cima=8
[Productor] Pila: [Ese es u--] cima=8

[Productor] Le: 'n'
[Productor] Inserido 'n' en pila[8] cima=9
[Productor] Pila: [Ese es un-] cima=9

[Productor] Le: ' '
[Productor] Inserido ' ' en pila[9] cima=10
[Productor] Pila: [Ese es un ] cima=10
```

Figura 1: Captura de parte de ejecución del programa en terminal.

El resultado más importante es que **el conteo final de vocales coincide** entre productor y consumidor, confirmando que ningún carácter se ha perdido ni duplicado. Además, el buffer no muestra guiones '-' siendo retirados como si fueran datos reales, a diferencia del apartado 1 donde eso era uno de los síntomas de las carreras críticas.

```
[Productor] Vogais lidas:
a=0 e=10 i=2 o=10 u=6

[Consumidor] Vogais consumidas:
a=0 e=10 i=2 o=10 u=6
```

Figura 2: Segunda captura de ejecución del programa en terminal.

La conclusión es que los semáforos son un mecanismo de sincronización: el protocolo `sem_wait/sem_post` es idéntico al del apartado 2, y solo cambia la forma de inicializarlos (`sem_init` en lugar de `sem_open`) y la forma de compartir los datos (variable global en lugar de `mmap`).